

Yesterday 11:54 AM

Lab_4_VibeCoding_Student_hand..
PDFVibeCodingGitRepoLinks.pdf
PDF

this is my assignment explai

Absolutely. I went through the assignment handout and the repository-list PDF. Here's what you actually have to do, in simple terms.

What is this assignment?

This is **Lab 4: Vibe Coding**.

The basic idea is:

You are given an existing Python game/puzzle that contains broken code. You use an AI coding tool such as ChatGPT, Gemini, or Claude to fix it and then add the features specified in that game's README.

The assignment is specifically testing whether you can **prompt an AI effectively to modify an existing codebase**, rather than manually coding everything yourself. The objective says you should aim to get the work done in roughly **3–4 prompting attempts**.  Lab_4_VibeCoding_Student_handou...

What you need to do

Step 1 — Find your assigned repository

Your second PDF contains **64 possible repositories**, each corresponding to a different game.

For example:

Repo	Game
01_frogger	Frogger
02_air_hockey	Air Hockey
09_tic_tac_toe	Tic Tac Toe
14_2048	2048
34_pacman	Pac-Man
40_flappy-bird	Flappy Bird
45_snake	Snake
46_space-invaders	Space Invaders
60_number_guess	Number Guess

The complete list is in your second PDF.  VibeCodingGitRepoLinks

You need to use the repository that was assigned to you, not choose whichever game you want.

Step 2 — Open the repository and read README

Once you know your repository, open it on GitHub.

The assignment explicitly says:

Go through the README file to understand the deliverables.

📄 Lab_4_VibeCoding_Student_handou...

This is **very important**.

The README should tell you:

1. What the existing game does
2. What is broken
3. What features you need to add
4. Possibly how to run it

So **don't start asking ChatGPT to change things before reading the README**.

Step 3 — Clone/Fork the repository

You need to get the assigned repository into your own GitHub account.

The assignment allows:

Clone OR Fork

📄 Lab_4_VibeCoding_Student_handou...

The overall workflow is basically:

```
graph TD
    A[SETAPESU26 repository] --> B[Clone/Fork]
    B --> C[Your GitHub repository]
    C --> D[Make changes]
    D --> E[Push changes]
```



Important

You **must NOT** create a Pull Request to the main SETAPESU26 repository.

The handout explicitly says:

Do not raise PR to the main repo (SETAPESU26)

📄 Lab_4_VibeCoding_Student_handou...

Step 4 — Run the game BEFORE changing anything

This is one of the important requirements.

You need to run the Python game in its **original/broken state**.

The assignment says to:

Run the Python based code (A game/A Puzzle)

and then:

Record 10 seconds of video before making any changes.

 Lab_4_VibeCoding_Student_handou...

So your first video should show something like:

```
Original game
↓
Run it
↓
Show the existing behaviour/bugs
↓
10-second recording
```

Do not modify the code before this recording.

Step 5 — Use Vibe Coding to fix the broken code

This is the main part of the assignment.

You use an AI coding tool.

For example, you could use ChatGPT and give it the project/code.

The assignment specifically says:

Write prompts and fix the broken code.

 Lab_4_VibeCoding_Student_handou...

The important part is that you're supposed to **prompt the AI**, rather than simply asking it generic questions.

For example, a good prompt would be something like:

"Analyze this Python game and identify all runtime errors and broken functionality. Fix the existing code without changing the intended gameplay. Run through the code carefully and provide the corrected files."

Then you test it.

If something is still broken, you make another prompt.

The assignment's objective specifically mentions trying to complete the task in **3–4 attempts**.

 Lab_4_VibeCoding_Student_handou...

Step 6 — Add ALL features mentioned in README

This is another major requirement.

After fixing the existing code, you need to look at the README and identify the features it asks you to a

Then:

Write prompts to add all the features listed in the README file one by one.

 Lab_4_VibeCoding_Student_handou...

So your workflow should look like:

Fix broken code

↓

Test

↓

Feature 1

↓

Test

↓

Feature 2

↓

Test

↓

Feature 3

↓

Test

You shouldn't just add random features that aren't requested.

Step 7 — Record the game AFTER everything is finished

Once you've:

- fixed the broken code
- added all README features
- tested everything

you record another **10-second video**.

The assignment requires:

Record another 10 seconds of video again after making all the changes.

📎 Lab_4_VibeCoding_Student_handou...

So you'll have:

Video 1

Before changes

Shows the original game.

Video 2

After changes

Shows your completed game.

Step 8 — Push your completed code

Once you're finished, push your code to **your repository**.

The assignment says:

Push the code to your repo.

📎 Lab_4_VibeCoding_Student_handou...

Again:

Do NOT make a PR to SETAPESU26.

Step 9 — Export your ChatGPT/AI chat history

This is easy to overlook.

You need to submit the **chat history** showing your Vibe Coding interaction.

The deliverables specifically require:

Chat history exported as a doc/pdf

 Lab_4_VibeCoding_Student_handou...

So your conversation should show things like:

You:
Analyze the code and fix the broken functionality...

AI:
Here's what is wrong...

You:
Now implement feature 1...

AI:
...

You:
Now implement feature 2...

AI:
...

This is essentially evidence that you actually used Vibe Coding.

Step 10 — Create Lab-4 folder in your repo

Your final repository should contain a **Lab-4 folder** with the required deliverables.

The handout says to upload the following under the **Lab-4 folder**:  Lab_4_VibeCoding_Student_handou...

Required:

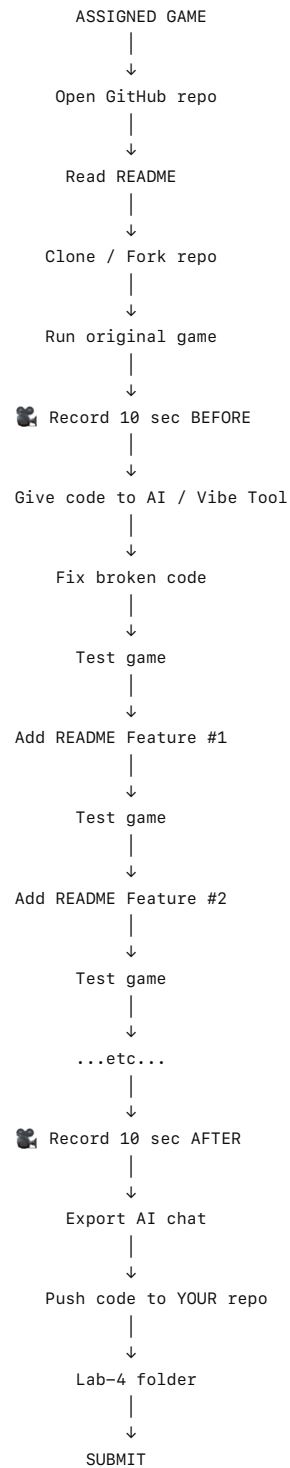
Lab-4/
├── Before video
├── After video
├── Updated code
└── Chat history PDF/DOC

The exact filenames aren't specified in the handout, so you can use sensible names such as:

Lab-4/
├── before.mp4
├── after.mp4
├── updated_code/
└── chat_history.pdf

Your complete workflow

Think of the entire assignment as this:



What you ultimately submit

Item	Required?
Original/before 10-sec video	✓
Final/after 10-sec video	✓
Updated code	✓
Exported AI chat history	✓
Lab-4 folder	✓
Push to your own repo	✓
Pull request to SETAPESU26	✗ Do not do this

The assignment is only **60 minutes**, so you should avoid spending too much time manually debugging or rewriting the project. [📄 Lab_4_VibeCoding_Student_handou...](#)

One thing I need from you

Tell me which repository/game was assigned to you (e.g. 34_pacman , 45_snake , 01_frogger), or send me a screenshot of your assigned repo.

Then I can take you through the assignment **step-by-step**, including:

1. how to clone it,
2. how to run it,
3. exactly what the README requires,
4. the prompts you should give ChatGPT,
5. how to make the changes,
6. and exactly what you need to submit.

Sources

main.py
Python

README.md
File

requirements.txt
Document

game_engine.py
Python

maze.py
Python

player.py
Python


```
dhaydb@dhaydb-MacBook-Pro-2 12_maze_runner % python3 main.py
Traceback (most recent call last):
  File "/Users/dhaydb/Documents/SE_Lab4/12_maze_runner/main.py", line 1, in <module>
    from game.game_engine import GameEngine
  File "/Users/dhaydb/Documents/SE_Lab4/12_maze_runner/game/game_engine.py", line 1, in <module>
    import pygame
ModuleNotFoundError: No module named 'pygame'
```

Page 22 of 84

its not letting me do anything now

Perfect 🍌 Task 4 is working, so all four README features are now implemented.

1. Commit Task 4 separately

Run:

```
</> Bash
git status
```

Then:

```
</> Bash
git add game/game_engine.py
git commit -m "Add difficulty tiers"
git push
```

Your commit history should now look roughly like:

1. Fix player wall collision
2. Add shortest path hint
3. Add fog of war
4. Add timer leaderboard
5. Add difficulty tiers

Don't combine these commits since you wanted each task committed separately.

2. Do one final full test

Before recording the AFTER video, verify:

- **Difficulty screen** appears when starting the game
- **Easy:** 10 × 8
- **Medium:** 15 × 13
- **Hard:** 20 × 18
- **R** generates a new maze at the **same difficulty**
- Player cannot walk through maze walls
- **H** toggles shortest-path hint
- Fog of War works
- Timer runs
- Reaching the exit completes the maze
- Completion time is saved to `leaderboard.json`
- Top 5 leaderboard is displayed
- Restarting the game retains the leaderboard

3. Record your AFTER video

You need the ~10-second recording showing the working version after your changes. I'd recommend showing:

Difficulty selection → select a difficulty → move around → press H → show fog/path functionality.

You don't need to demonstrate every feature in 10 seconds.

4. Then we finish the submission folder

The assignment requires the `Lab-4` folder to contain the **before/after videos, updated code, and exported chat history**.  `Lab_4_VibeCoding_Student_handou...`

Once you've made the Task 4 commit, tell me **"committed"** and I'll walk you through the final `Lab-4` folder setup, AFTER video, chat export, and final Git push.

[Sources](#)[commit](#)

Great. Now let's finish the submission cleanly.

Step 1 — Record the AFTER video

Run your game:

```
</> Bash
python main.py
```

Record about **10 seconds** showing the completed version.

A good sequence is:

1. Show the **difficulty selection screen**
2. Select a difficulty
3. Move the player around
4. Press **H** to show the shortest-path hint
5. Briefly show the fog-of-war effect

You don't need to demonstrate every feature in the video.

Step 2 — Create the Lab-4 folder

Inside your project, create:

```
Lab-4/
├── before.mp4
├── after.mp4
├── chat_history.pdf
└── code/
```

Put your **before video** and **after video** there.

For the code, copy the final project code into `Lab-4/code/`, or if your instructor expects the whole project, copy the relevant project files there.

The assignment specifically requires the `Lab-4` folder to contain the before/after videos, updated code and exported chat history. [Lab_4_VibeCoding_Student_handou...](#)

Step 3 — Export this chat

You need the chat history showing your Vibe Coding process, including:

- Initial bug-fixing prompt
- Player collision fix
- Task 1 prompt/implementation
- Task 2 prompt/implementation
- Task 3 prompt/implementation
- Task 4 prompt/implementation
- Testing/verification

Export/save the conversation as a **PDF** and put it in:

Lab-4/chat_history.pdf

Step 4 — Final Git check

After adding the submission files:

```
</> Bash
git status
```

Then:

```
</> Bash
git add Lab-4
git commit -m "Add Lab-4 submission files"
git push
```

Final structure

You should end up with something like:

```
12_maze_runner/
├── game/
│   ├── game_engine.py
│   ├── maze.py
│   └── player.py
├── main.py
├── requirements.txt
├── README.md
├── leaderboard.json
└── Lab-4/
    ├── before.mp4
    ├── after.mp4
    ├── chat_history.pdf
    └── code/
        ├── main.py
        ├── requirements.txt
        ├── README.md
        ├── leaderboard.json
        └── game/
            ├── game_engine.py
            ├── maze.py
            └── player.py
```

One important thing: don't accidentally modify `leaderboard.json` after your final test and commit a bunch of unwanted scores. The important thing is that the code demonstrates that the leaderboard persists and maintains the top 5.

Once you've recorded the **AFTER video**, we can do the final folder/commit check.

Sources

how do I export this cha

Since you need a **PDF of this specific chat** for your assignment, you don't need the full ChatGPT data export. The easiest way on your Mac is to use the browser's **Print → Save as PDF**.

Do this:

1. Stay on **this conversation**.
2. Press:
 ⌘ **Command + P**
3. In the print window, choose **PDF** at the bottom-left.
4. Select **Save as PDF**.
5. Name it:
 chat_history.pdf
6. Save it somewhere you can find easily.
7. Move it into your project:

Lab-4/
└─ chat_history.pdf



This gives you a PDF containing the visible conversation, which is much more suitable for your assignment than the official **Export Data** feature.

The official ChatGPT data export is different: **Settings → Data Controls → Export Data** produces a ZIP containing your chat history and other account data, and OpenAI says the export can take up to 7 days.

help.openai.com

So for your assignment: use ⌘P → Save as PDF.

